

CONTENTS

Práctico 11 (Pipeline, Jerarquía de Memoria) — Arquitectura de Computadoras	
FING UdelaR · Resolución completa y verificada	
Preguntas teóricas	
Ejercicio 1 — Localidad en un recorrido anidado	
Ejercicio 2 — Programa con poca localidad temporal y mucha localidad espacial	
Ejercicio 3 — Esquemas de dirección (caché de 4 KB, 256 líneas, direcciones de 20 bits)	
Ejercicio 4 — Caché asociativa por conjuntos de 4 vías (32 KB, direcciones de 32 bits)	
Ejercicio 5 — Hit rate según recorrido de matrices (caché de 128 KB, líneas de 32 B, directa)	
(a) División de la dirección de memoria	
(b) Tasa de aciertos del código dado	
(c) ¿Mejora recorrer primero columnas y luego filas? ¿Y si además se almacenan por columnas?	
(d) Diseño de caché que mejore la tasa de aciertos sin costo excesivo de búsqueda	
Ejercicio 6 — Jerarquía L1/L2 (CPU de 16 bits, direcciones de 24 bits)	
(a) Interpretación de la dirección para la caché L1	
(b) Bytes transferidos entre memoria principal y caché ante un miss en L1	
(c) Interpretación de la dirección para la caché L2	
(d) Hit en L1: bytes transferidos CPU↔L1 y ciclos de bus	
(e) Miss en L1, hit en L2: bytes transferidos L1↔L2 y ciclos de bus	
(f) Miss en L1 y en L2: bytes transferidos L2↔memoria y ciclos de bus	

Resumen de técnicas usadas en este práctico

.....

Práctico 11 (Pipeline, Jerarquía de Memoria) — Arquitectura de Computadoras

FING UDELAR · RESOLUCIÓN COMPLETA Y VERIFICADA

⚠ **Nota general:** el material original del curso ([Ejercicios Resueltos.pdf](#)) sólo llega hasta el Ejercicio 5(a) — la pregunta teórica (g), los incisos (b), (c) y (d) del Ejercicio 5, y el Ejercicio 6 completo (a-f) **no estaban resueltos**. Todos esos puntos se resuelven acá desde cero. Cada cálculo de descomposición de dirección se verificó sumando TAG+LÍNEA/CONJUNTO+BYTE contra el total de bits de dirección, y los cálculos de tasa de aciertos del Ejercicio 5 se verificaron simulando el recorrido real en Python (celda por celda, con las direcciones reales de a, b y c) en vez de estimarlos a ojo — el resultado de ese ejercicio es contra-intuitivo y no sale bien si se estima “de memoria”.

Preguntas teóricas

(a) ¿Qué es un Hazard? ¿Cómo se clasifican?

Un *hazard* es cualquier situación que impide que el pipeline avance de la manera ideal (una instrucción nueva entrando por ciclo, sin detenerse). Se clasifican en tres grupos:

- **Estructurales:** dos etapas distintas del pipeline, en el mismo ciclo, necesitan el mismo recurso físico de hardware (ej. memoria compartida entre Fetch y Read/Memory, o una única ALU compartida entre el cálculo de PC+4 y una operación aritmética).
- **De datos:** una instrucción depende de un dato que otra instrucción, todavía dentro del pipeline, no terminó de producir (o va a sobrescribir/leer en el orden incorrecto). Se subdividen en RAW (Read After Write, el más común), WAR (Write After Read) y WAW (Write After Write).
- **De control:** los saltos (condicionales o incondicionales) rompen el supuesto de que “la próxima instrucción a buscar es la siguiente en memoria”, obligando a descartar trabajo ya adelantado por el pipeline si la suposición resultó incorrecta.

(b) Forwarding (register bypassing).

Es la técnica más usada para mitigar hazards RAW sin necesidad de detener el pipeline. En vez de esperar a que el resultado de una instrucción productora pase por su etapa *Write* y quede guardado en el banco de registros para recién ahí poder leerlo, se agrega hardware adicional (un multiplexor a la entrada de la ALU) que toma el resultado **apenas sale de la ALU** (al

final de la etapa *Execute* de la instrucción productora) y lo entrega directamente a la entrada de la etapa que lo necesita (típicamente la etapa *Execute* de la instrucción siguiente), sin pasar por el banco de registros. Esto evita el stall en la mayoría de los casos.

⚠ El forwarding **mitiga** el hazard RAW, no lo elimina en todos los casos: si el dato depende de un acceso lento a memoria (una instrucción *load*), puede seguir haciendo falta al menos una burbuja de un ciclo, porque el dato físicamente no existe todavía en el momento en que se lo necesita, por más que haya forwarding.

(c) Técnica para reducir hazards de control.

Una técnica típica es el **prefetch de ambos destinos posibles**: ante un salto, se hace simultáneamente el fetch de la instrucción siguiente en memoria (secuencia normal) y el fetch de la instrucción en la dirección de destino del salto. Cuando finalmente se resuelve si el salto se toma o no, se descarta la instrucción que no corresponda a la secuencia lógica real, sin haber perdido tiempo adicional de fetch esperando a saber el resultado.

Otras técnicas alternativas (cualquiera es válida como respuesta): **salto demorado** (delayed branch: siempre se ejecuta la instrucción inmediatamente siguiente al salto, reordenada por el compilador para que sea útil — nunca hay descarte de trabajo), **múltiples flujos especulativos** (duplicar hardware para avanzar por ambos caminos posibles a la vez) y **predicción de salto** (estática o dinámica, con tablas de historia de saltos).

(d) Correspondencia directa. Políticas de reemplazo aplicables.

Dada una dirección de memoria se distinguen tres campos: **TAG**, **LÍNEA** y **BYTE** (desplazamiento). El campo LÍNEA indica el único índice de línea de la caché donde puede alojarse un bloque de memoria dado — es una relación 1 a 1 entre bloque y línea candidata (aunque muchos bloques distintos compitan por esa misma línea a lo largo del tiempo). Para determinar hit/miss: se usa LÍNEA para saber en qué línea mirar, y ahí se compara el TAG almacenado contra el TAG de la dirección buscada.

Como cada bloque tiene una única línea destino posible, **no aplica ninguna política de reemplazo**: no hay elección que hacer, si la línea candidata está ocupada por otro bloque simplemente se desaloja y se coloca el nuevo.

(e) Correspondencia asociativa por conjuntos. Políticas de reemplazo aplicables.

Los campos de la dirección son **TAG**, **CONJUNTO** y **BYTE**. Las líneas se agrupan en conjuntos de N líneas (“N vías”); un bloque de memoria tiene un único conjunto candidato, pero dentro de ese conjunto puede ir a **cualquiera** de las N líneas. Para determinar hit/miss: se usa

CONJUNTO para saber en qué conjunto mirar, y ahí se comparan los N TAGs almacenados contra el TAG de la dirección buscada — si alguno coincide, hit; si no, miss.

Como dentro de un conjunto sí hay elección posible entre N líneas, **si aplican políticas de reemplazo**: por ejemplo **LRU** (Least Recently Used — desaloja la línea que hace más tiempo que no se usa, la que mejor aprovecha localidad temporal pero más cara de implementar exactamente), **FIFO** (desaloja la que entró primero al conjunto, más simple), **LFU** (desaloja la de menor frecuencia de uso) o **Random** (al azar, trivial en hardware).

(f) Write-through vs. Write-back.

Ambas son políticas que definen qué pasa cuando la CPU escribe un dato que produce hit de escritura en caché:

- **Write-through**: la escritura se realiza **simultáneamente** en la caché **y** en la memoria principal, siempre. Ventaja: la memoria principal siempre queda consistente con la caché. Desventaja: cada escritura paga el costo de acceso a memoria principal aunque haya sido hit, lo que es más lento.
- **Write-back**: la escritura se realiza **solo en la caché**, marcando la línea con un bit adicional **dirty** (modificado). La memoria principal recién se actualiza cuando esa línea se desaloja de la caché, y sólo si el bit dirty está en 1. Ventaja: mucho más rápida (no genera tráfico a memoria en cada escritura). Desventaja: necesita el bit dirty extra y la memoria principal puede quedar temporalmente desactualizada respecto a la caché.

(g) Problema de coherencia de caché. ¿Cómo se resuelve?

 Este inciso no estaba resuelto en el material original — se resuelve acá desde cero.

En un sistema con varios procesadores (o núcleos), cada uno con su propia caché privada, puede ocurrir que **dos cachés distintas tengan copias del mismo bloque de memoria principal**. Si un procesador modifica su copia local (sobre todo bajo Write-back, donde la modificación puede quedar únicamente en su caché por un buen rato sin propagarse a memoria), las demás cachés quedan con una copia **desactualizada** de ese mismo bloque sin saberlo — a esto se le llama **problema de coherencia de caché**: distintos procesadores podrían ver valores distintos e inconsistentes de la misma dirección de memoria.

Solución típica: Bus Snooping (“espionaje de bus”). Cada caché agrega lógica que escucha permanentemente el bus de memoria compartido, observando todas las transacciones de escritura que hacen las demás cachés. Si una caché detecta, por el bus, que otro procesador escribió un bloque que ella también tiene almacenado, **invalida su propia copia** de ese bloque (no la actualiza con el valor nuevo, simplemente la marca como inválida). La próxima vez que

ese procesador necesite el dato, se produce un miss y se trae la versión actualizada desde memoria (o, en protocolos más sofisticados como MESI, directamente desde la caché que la modificó).

Ejercicio 1 — Localidad en un recorrido anidado

```
for (i = 0; i < 20; i++)
  for (j = 0; j < 10; j++)
    a[i] = a[i] * a[j]
```

(a) Localidad espacial: para un `i` fijo, a medida que `j` avanza (0,1,2,...,9) se accede secuencialmente a `a[j]`, `a[j+1]`, `a[j+2]`, ... — es decir, a posiciones de memoria consecutivas, el ejemplo clásico de recorrido secuencial de un arreglo.

(b) Localidad temporal: para un `i` fijo, la celda `a[i]` se **lee y escribe** en cada una de las 10 vueltas del bucle interno (`j=0..9`) — se vuelve a referenciar al menos 10 veces seguidas en un intervalo corto de tiempo. Concretamente, tras las 10 iteraciones queda:

$$a[i] = a[i]_{\text{inicial}} \cdot a[0] \cdot a[1] \cdot \dots \cdot a[9]$$

donde `a[i]` fue leído y escrito 10 veces consecutivas.

Ejercicio 2 — Programa con poca localidad temporal y mucha localidad espacial

Características generales: un programa así accede, en cada paso, a una posición **cercana** a la anterior (buena localidad espacial), pero **nunca vuelve** a una posición ya visitada al poco tiempo (mala localidad temporal) — el “frente de acceso” avanza (o se contrae) por la memoria sin repetirse.

Ejemplo: búsqueda binaria.

```

BusquedaBinaria(a[], x):
    izq = 0
    der = n - 1
    mientras izq <= der:
        medio = (izq + der) / 2
        si a[medio] == x:
            retornar medio
        si no si a[medio] > x:
            der = medio - 1
        si no:
            izq = medio + 1
    retornar -1

```

- **Escasa localidad temporal:** el espacio de búsqueda se reduce a la mitad en cada iteración, por lo que casi nunca se vuelve a acceder a una posición ya visitada poco tiempo después.
- **Alta localidad espacial:** aunque el acceso no es estrictamente secuencial (`a[medio]` salta), en cada iteración `medio` queda dentro del rango `[izq, der]` que se va acotando, así que iteraciones consecutivas acceden a posiciones cada vez más cercanas entre sí (la distancia `der-izq` se reduce a la mitad en cada paso).

Ejercicio 3 — Esquemas de dirección (caché de 4 KB, 256 líneas, direcciones de 20 bits)

Datos comunes: caché = 4 KB = 2^{12} bytes, 256 = 2^8 líneas → tamaño de línea = $2^{12}/2^8 = 2^4 = 16$ bytes/línea → **BYTE = 4 bits** (esto no depende del tipo de correspondencia).

(a) Correspondencia directa. LÍNEA = $\log_2(256) = 8$ bits. TAG = $20 - 4 - 8 = 8$ bits.

Verificación: $8+8+4 = 20$ ✓ TAG[19..12] | LÍNEA[11..4] | BYTE[3..0]

(b) Correspondencia totalmente asociativa. No existe campo LÍNEA (cualquier bloque puede ir a cualquier línea). TAG = $20 - 4 = 16$ bits. Verificación: $16+4 = 20$ ✓ TAG[19..4] |

BYTE[3..0]

(c) Correspondencia asociativa por conjuntos de 2 vías. Conjuntos = $256/2 = 128 = 2^7$ →

CONJUNTO = 7 bits. TAG = $20 - 4 - 7 = 9$ bits. Verificación: $9+7+4 = 20$ ✓ TAG[19..11] |

CONJUNTO[10..4] | BYTE[3..0]

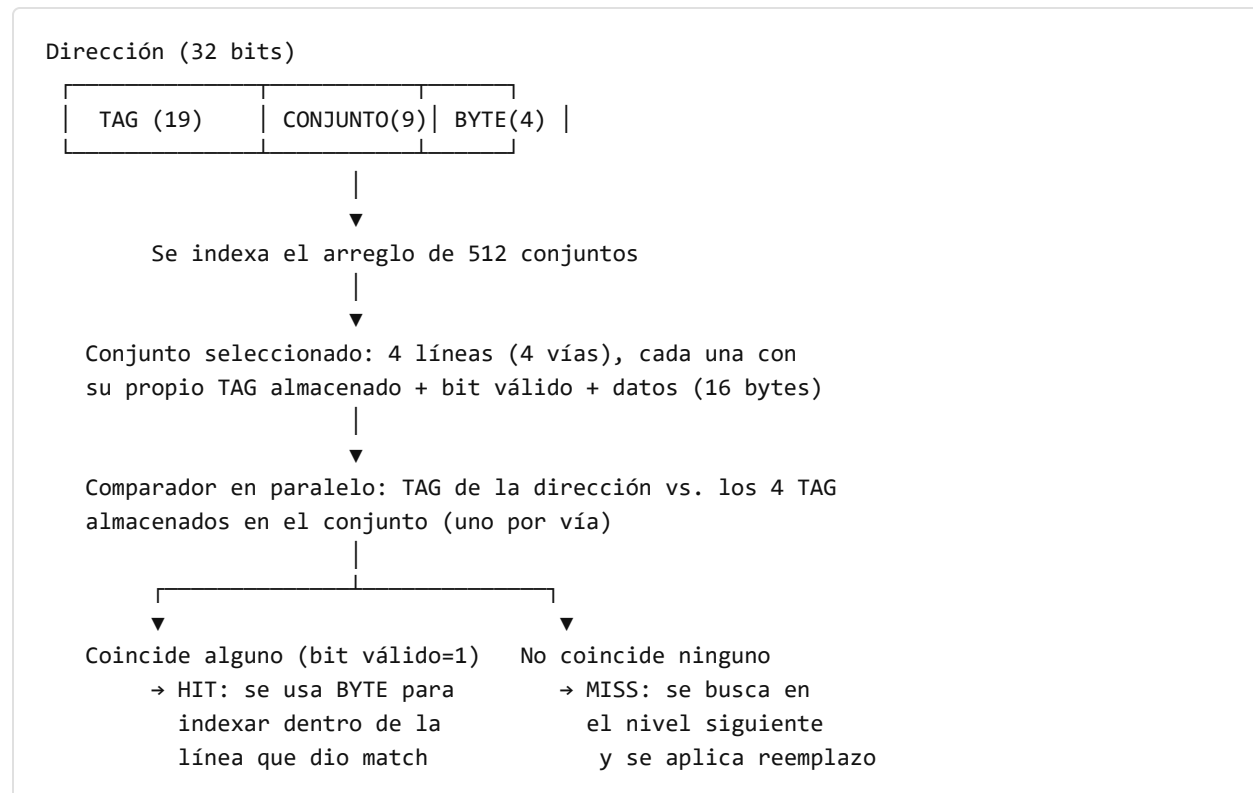
Ejercicio 4 — Caché asociativa por conjuntos de 4 vías (32 KB, direcciones de 32 bits)

Caché = 32 KB = 2^{15} bytes. Línea = 4 palabras de 32 bits = 16 bytes = 2^4 → **BYTE = 4 bits**.

Cantidad de líneas = $2^{15}/2^4 = 2^{11} = 2048$ líneas. Conjuntos = $2048/4$ vías = $512 = 2^9$ →

CONJUNTO = 9 bits. TAG = $32 - 4 - 9 = 19$ bits. Verificación: $19+9+4 = 32$ ✓ TAG[31..13] | CONJUNTO[12..4] | BYTE[3..0]

Diagrama de bloques (organización lógica):



¿Dónde se asigna la palabra de la dirección 0xABCDE8F8?

0xABCDE8F8 en binario (32 bits): 1010 1011 1100 1101 1110 1000 1111 1000

Separando según el esquema TAG[31..13] | CONJUNTO[12..4] | BYTE[3..0]:

- TAG = 10101011111001101111 (19 bits)
- CONJUNTO = 010001111 (9 bits) = $128+8+4+2+1 = 143$
- BYTE = 1000 (4 bits) = 8

(Verificado con Python: `format(0xABCDE8F8, '032b')` seccionado exactamente en esos 3 campos da CONJUNTO=143, BYTE=8.)

Conclusión: la palabra de esa dirección se busca en el **conjunto 143**, comparando el TAG contra las 4 líneas (vías) de ese conjunto, y dentro de la línea que dé match, en el **byte 8**.

Ejercicio 5 — Hit rate según recorrido de matrices (caché de 128 KB, líneas de 32 B, directa)

Datos: direcciones de 32 bits, memoria direccionable por byte, caché de 128 KB, líneas de 32 bytes, correspondencia directa. Código:

```
for (i = 0; i < 512; i++)
    for (j = 0; j < 512; j++)
        c[i][j] = a[i][j] + b[i][j]
```

a, b, c son matrices de 512×512 enteros, almacenadas **de forma consecutiva** en memoria, cada una por filas (row-major). a comienza en 0x00100000.

(A) DIVISIÓN DE LA DIRECCIÓN DE MEMORIA

Caché = 128 KB = 2^{17} bytes. Línea = 32 bytes = 2^5 → **BYTE = 5 bits**. Cantidad de líneas = $2^{17}/2^5 = 2^{12} = 4096$ líneas → **LÍNEA = 12 bits**. TAG = 32 – 5 – 12 = **15 bits**. Verificación: $15+12+5 = 32$ ✓ TAG[31..17] | LÍNEA[16..5] | BYTE[4..0]

Justificación: con correspondencia directa siempre son 3 campos (TAG/LÍNEA/BYTE); BYTE sale del tamaño de línea (32 B ⇒ 5 bits para direccionar cada byte dentro de la línea), LÍNEA sale de la cantidad de líneas físicas de la caché (4096 ⇒ 12 bits para indexarlas), y TAG se lleva los bits restantes de la dirección de 32 bits.

(B) TASA DE ACIERTOS DEL CÓDIGO DADO

⚠ Nota de transparencia sobre supuestos. El enunciado no da el tamaño de “entero” ni la dirección exacta de b y c — se asume **entero = 4 bytes** (estándar en esta arquitectura de 32 bits) y que, al decir “se almacenan en memoria de forma consecutiva” con a arrancando en 0x00100000, b comienza inmediatamente después de a, y c inmediatamente después de b. Cada matriz ocupa $512 \times 512 \times 4 = 1.048.576$ bytes = 2^{20} = **exactamente 0x00100000 bytes** (1

MB) — un número redondo que además coincide exactamente con que a arranque en un límite de 1 MB (0x00100000), lo cual es la señal de que el enunciado está diseñado a propósito con este alineamiento. Bajo ese supuesto:

- a: 0x00100000 – 0x001FFFFFF
- b: 0x00200000 – 0x002FFFFFF
- c: 0x00300000 – 0x003FFFFFF

El punto clave (y la “trampa” de este ejercicio): el tamaño de cada matriz ($1 \text{ MB} = 2^{20}$) es un múltiplo exacto del tamaño de la caché ($128 \text{ KB} = 2^{17}$): $2^{20}/2^{17} = 8$. Como el campo LÍNEA de una correspondencia directa depende únicamente de `dirección mod tamaño_caché`, y las tres bases (0x100000, 0x200000, 0x300000) difieren entre sí en múltiplos exactos de 2^{17} , se cumple que para **cualquier** posición (i,j):

$$(\text{dirección de } a[i][j]) \bmod 2^{17} = (\text{dirección de } b[i][j]) \bmod 2^{17} = (\text{dirección de } c[i][j]) \bmod 2^{17}$$

Es decir: **$a[i][j]$, $b[i][j]$ y $c[i][j]$ siempre mapean a la misma línea de la caché**, sin importar qué (i,j) se esté procesando.

Como el ciclo interno hace, para cada (i,j), 3 accesos consecutivos — lectura de $a[i][j]$, lectura de $b[i][j]$, escritura de $c[i][j]$ (la nota del enunciado aclara que una escritura también es un acceso a memoria) — y los tres compiten por la **misma línea física** en una caché de correspondencia directa (donde sólo una puede estar presente a la vez), el acceso a $b[i][j]$ desaloja la línea recién cargada por $a[i][j]$, y el acceso a $c[i][j]$ desaloja a su vez la de $b[i][j]$. En la iteración siguiente (j+1), el acceso a $a[i][j+1]$ ya no encuentra en caché la línea que había traído $a[i][j]$ (aunque en teoría esos 4 bytes de a estén en la misma línea de 32 B que otros 7 enteros de a) porque esa línea fue desalojada por b y luego por c antes de poder reutilizarse.

Resultado: todos los accesos son miss.

- Total de accesos: $512 \times 512 \times 3 = 786.432$ (lectura de a, lectura de b, escritura de c, por cada una de las 512×512 celdas).
- Misses: 786.432. Hits: 0.
- **Tasa de aciertos = $0/786.432 = 0\%$.**

Verificación por simulación en Python (caché directa de 4096 líneas de 32 B, direcciones reales 0x00100000/0x00200000/0x00300000, recorrido i-luego-j exactamente como en el código):

`hits=0, misses=786432, hit_rate=0.0` ✓. Se simuló accediendo exactamente a las direcciones reales, no una estimación.

⚠ Este resultado es **contra-intuitivo**: la sola inspección del código (recorrido por filas, que coincide con el almacenamiento por filas) sugeriría alta localidad espacial y un hit rate alto (como en el Ejemplo de la Sección 7 de la guía de jerarquía de memoria, con una sola matriz). Pero acá hay **tres** arreglos distintos entrelazados en cada iteración, y su alineamiento de direcciones (offset entre bases = múltiplo exacto del tamaño de caché) hace que se pisen entre sí en cada acceso. Este es precisamente el “truco” que hace interesante al ejercicio.

(C) ¿MEJORA RECORRER PRIMERO COLUMNAS Y LUEGO FILAS? ¿Y SI ADEMÁS SE ALMACENAN POR COLUMNAS?

Caso 1 — recorrido por columnas (for j: for i), matrices siguen almacenadas por filas: no mejora. El conflicto de línea entre $a[i][j]$, $b[i][j]$ y $c[i][j]$ ocurre **para cada (i,j) individual**, sin importar en qué orden se visiten los pares (i,j) — el problema no es “en qué orden recorro la matriz” sino que las tres direcciones de una misma celda (i,j) siempre caen en la misma línea física. Cambiar el orden de recorrido no cambia esa relación entre direcciones. Además, recorrer por columnas cuando el almacenamiento es por filas **agrega** un segundo problema (destruye la localidad espacial dentro de cada matriz individual: $a[i][j]$ y $a[i+1][j]$ distan 2048 bytes, muy lejos dentro de la misma línea de 32 B), pero como ya el hit rate era 0% por el problema anterior, no hay margen para empeorar.

Caso 2 — recorrido por columnas y matrices almacenadas también por columnas (column-major): tampoco mejora. Almacenar por columnas y recorrer por columnas sí restaura la localidad espacial **dentro de cada matriz** (ahora $a[i][j]$ y $a[i+1][j]$ vuelven a ser consecutivos en memoria), pero el conflicto entre a, b y c **no depende del layout interno de cada matriz**, sino únicamente de que sus direcciones base difieren en un múltiplo exacto del tamaño de la caché. Ese desplazamiento entre bases (1 MB) se mantiene igual sea cual sea el orden de almacenamiento interno de cada matriz. Por lo tanto, $a[i][j]$, $b[i][j]$ y $c[i][j]$ siguen aliasing en la misma línea en cada acceso, y el hit rate sigue siendo 0%.

Verificación por simulación en Python de ambas variantes: recorrido por columnas con almacenamiento por filas → hits=0, misses=786432, hit_rate=0.0. Recorrido por columnas con almacenamiento por columnas → hits=0, misses=786432, hit_rate=0.0. Ambos coinciden exactamente con el caso original (0%). ✓

Conclusión: ninguno de los dos cambios propuestos mejora la tasa de aciertos, porque el problema real no es de orden de recorrido ni de layout de almacenamiento, sino de **alineamiento de las direcciones base** de las tres matrices respecto al tamaño de la caché en una correspondencia **directa** (sin asociatividad no hay forma de que las tres líneas convivan simultáneamente).

(D) DISEÑO DE CACHÉ QUE MEJORE LA TASA DE ACIERTOS SIN COSTO EXCESIVO DE BÚSQUEDA

Propuesta: cambiar la correspondencia de **directa** a **asociativa por conjuntos de 4 vías**, manteniendo el mismo tamaño total de caché (128 KB) y el mismo tamaño de línea (32 B). Esto multiplica por 4 la cantidad de líneas que pueden convivir en el mismo conjunto sin desalojarse entre sí, alcanzando para que las 3 líneas en conflicto (a, b y c) quepan simultáneamente (basta con que las vías sean ≥ 3 ; se elige 4 por ser potencia de 2, la opción de hardware habitual).

¿Por qué “sin incremento excesivo”? Al mantener el mismo tamaño total y línea, sólo se agrega la lógica de comparación en paralelo de 4 TAGs por conjunto (en vez de 1), que es exactamente el mismo costo que ya calculamos en el Ejercicio 4 — factible en hardware real y mucho más barato que una totalmente asociativa (que compararía contra las 4096 líneas enteras).

Nuevo esquema de dirección: conjuntos = $4096 \text{ líneas} / 4 \text{ vías} = 1024 = 2^{10} \rightarrow \text{CONJUNTO} = 10 \text{ bits}$. TAG = $32 - 5 - 10 = 17 \text{ bits}$. Verificación: $17+10+5 = 32 \checkmark$ TAG[31..15] | CONJUNTO[14..5] | BYTE[4..0]

Por qué funciona: el offset entre las bases de a, b y c ($1 \text{ MB} = 2^{20}$) sigue siendo múltiplo del tramo cubierto por los conjuntos ($2^{10} \text{ conjuntos} \times 32 \text{ B} = 2^{15} = 32 \text{ KB}$; $2^{20}/2^{15}=32$, exacto), así que $a[i][j]$, $b[i][j]$ y $c[i][j]$ **siguen** mapeando al mismo conjunto — pero ahora ese conjunto tiene 4 líneas disponibles, alcanza para alojar simultáneamente la línea de a, la de b y la de c sin que se desalojen entre sí. Dentro de un bloque de 8 valores consecutivos de j (ya que la línea de 32 B guarda 8 enteros), sólo el primer j del bloque genera miss para cada una de las 3 matrices (falta cargar la línea nueva); los 7 j restantes del bloque son hit en las 3 matrices, porque sus líneas siguen residentes en el conjunto.

- Por cada bloque de 8 valores de j: $8 \times 3 = 24$ accesos, de los cuales 3 son miss (el primer acceso a a, b y c del bloque) y 21 son hit.
- Tasa de aciertos = $21/24 = 87,5\%$.

Verificación por simulación en Python (caché asociativa por conjuntos con política LRU, 4 vías, mismas direcciones reales): con 1 vía (directa) $\rightarrow \text{hit_rate}=0.0$ (coincide con (b)); con 2 vías $\rightarrow \text{hit_rate}=0.0$ (2 vías no alcanzan para 3 líneas en conflicto); con 3 vías $\rightarrow \text{hit_rate}=0.875$; con 4 vías $\rightarrow \text{hit_rate}=0.875$; con 8 vías $\rightarrow \text{hit_rate}=0.875$ (no sigue mejorando, el cuello de botella ya no es la asociatividad sino que cada línea nueva de 32 B trae 8 enteros y siempre hay 1 miss “de arranque” cada 8 accesos por matriz). El salto ocurre exactamente al pasar de 2 a 3 vías, confirmando que 3 es el mínimo necesario para alojar a, b y c simultáneamente — 4 vías (potencia de 2) es la elección práctica. $\checkmark$

Ejercicio 6 — Jerarquía L1/L2 (CPU de 16 bits, direcciones de 24 bits)

⚠ Este ejercicio completo no estaba resuelto en el material original — se resuelve acá desde cero, verificando cada cálculo con Python.

Datos: CPU de 16 bits, direcciones de 24 bits. Caché L1: 1 KB, 16 líneas, totalmente asociativa. Luego se agrega caché L2: 32 KB, 128 líneas, asociativa por conjuntos de 4 vías. Buses de datos de 16 bits (CPU↔L1, L1↔L2, L2↔memoria), todos a la misma frecuencia. Los pedidos de un nivel superior a uno inferior son a nivel de bloque (se pide/transfiere la línea completa del nivel que hace el pedido).

(A) INTERPRETACIÓN DE LA DIRECCIÓN PARA LA CACHÉ L1

$L1 = 1 \text{ KB} = 2^{10} \text{ bytes}$, $16 = 2^4 \text{ líneas} \rightarrow \text{tamaño de línea} = 2^{10}/2^4 = 2^6 = \mathbf{64 \text{ bytes/línea}} \rightarrow \mathbf{BYTE = 6 \text{ bits}}$.

Totalmente asociativa $\Rightarrow$ no existe campo LÍNEA (cualquier bloque puede ir a cualquier línea de las 16). TAG = $24 - 6 = \mathbf{18 \text{ bits}}$.

Verificación: $18+6 = 24 \checkmark$ TAG[23..6] | BYTE[5..0]

Parámetros relevantes: 16 líneas, 64 B/línea, comparación en paralelo del TAG (18 bits) contra las 16 líneas simultáneamente.

(B) BYTES TRANSFERIDOS ENTRE MEMORIA PRINCIPAL Y CACHÉ ANTE UN MISS EN L1

En este punto (antes de agregar L2) un miss en L1 va directo a memoria principal. Como el pedido es a nivel de bloque, se transfiere la **línea completa** de L1: **64 bytes**.

(C) INTERPRETACIÓN DE LA DIRECCIÓN PARA LA CACHÉ L2

$L2 = 32 \text{ KB} = 2^{15} \text{ bytes}$, $128 = 2^7 \text{ líneas} \rightarrow \text{tamaño de línea de L2} = 2^{15}/2^7 = 2^8 = \mathbf{256 \text{ bytes/línea}} \rightarrow \mathbf{BYTE = 8 \text{ bits}}$ (distinto del BYTE de L1: cada nivel de caché interpreta la misma dirección física con sus **propios** campos, según su propio tamaño de línea).

⚠ Nota de transparencia: el enunciado no dice explícitamente que el tamaño de línea de L2 deba calcularse de forma independiente a partir de sus propios 32 KB/128 líneas — se asume esta lectura estándar (cada nivel de la jerarquía tiene su propio tamaño de línea, determinado

por su propia capacidad y cantidad de líneas), que es además la interpretación consistente con que L2 sea más grande y normalmente tenga líneas más grandes que L1 en jerarquías reales (acá 256 B vs. 64 B, 4 veces más grande).

Asociativa por conjuntos de 4 vías $\Rightarrow$ conjuntos = $128/4 = 32 = 2^5 \rightarrow$ **CONJUNTO = 5 bits**. TAG = $24 - 8 - 5 = 11$ bits.

Verificación: $11+5+8 = 24 \checkmark$ TAG[23..13] | CONJUNTO[12..8] | BYTE[7..0]

Parámetros relevantes: 32 conjuntos, 4 vías por conjunto, 256 B/línea, comparación en paralelo de 4 TAGs (11 bits cada uno) dentro del conjunto seleccionado.

(D) HIT EN L1: BYTES TRANSFERIDOS CPU $\leftrightarrow$ L1 Y CICLOS DE BUS

La CPU es de 16 bits, así que accede a la caché en unidades de **1 palabra = 2 bytes**. Ante un hit en L1, se transfieren **2 bytes** entre la CPU y L1.

Con bus de datos de 16 bits (2 bytes por ciclo de bus) entre CPU y L1: transferir 2 bytes toma **1 ciclo de bus** (2 bytes / 2 bytes-por-ciclo = 1).

(E) MISS EN L1, HIT EN L2: BYTES TRANSFERIDOS L1 $\leftrightarrow$ L2 Y CICLOS DE BUS

Como los pedidos desde un nivel superior son a nivel de bloque, ante un miss en L1 se pide a L2 el **bloque completo de L1** (no sólo la palabra) — es decir, **64 bytes** (el tamaño de línea de L1, calculado en (a)).

Con bus de 16 bits (2 bytes/ciclo) entre L1 y L2: 64 bytes / 2 bytes-por-ciclo = **32 ciclos de bus**.

(F) MISS EN L1 Y EN L2: BYTES TRANSFERIDOS L2 $\leftrightarrow$ MEMORIA Y CICLOS DE BUS

Si además hay miss en L2, L2 debe traer desde memoria principal su **propia línea completa** (no sólo los 64 bytes que necesitaba L1) — es decir, **256 bytes** (el tamaño de línea de L2, calculado en (c)). Esto es “sobre-búsqueda” respecto de lo que pidió L1, pero es como operan realmente las cachés: siempre rellenan su propia granularidad de línea, sea cual sea el tamaño del pedido que las originó.

Con bus de 16 bits (2 bytes/ciclo) entre L2 y memoria: 256 bytes / 2 bytes-por-ciclo = **128 ciclos de bus**.

Verificación de todos los cálculos de este ejercicio con Python (log2 de tamaños/cantidades de líneas, y división bytes/bus_ancho para los ciclos): L1 → line=64B, byte_bits=6, tag_bits=18 (suma 24 ✓). L2 → line=256B, byte_bits=8, sets=32, set_bits=5, tag_bits=11 (suma 24 ✓). Ciclos: CPU-L1 hit = $2/2 = 1.0$ ciclo; L1 miss/L2 hit = $64/2 = 32.0$ ciclos; L1 y L2 miss = $256/2 = 128.0$ ciclos. Todos los valores coinciden con los presentados arriba. ✓

Resumen de técnicas usadas en este práctico

Ejercicio	Técnica
Teóricas (a)-(g)	Clasificación de hazards, forwarding, técnicas anti-hazard de control, organización directa/por conjuntos, write-through/write-back, coherencia por bus snooping
1, 2	Identificación de localidad temporal/espacial en código con bucles anidados y en búsqueda binaria
3, 4, 5(a), 6(a,c)	Descomposición de dirección TAG/LÍNEA(o CONJUNTO)/BYTE, verificando siempre que la suma de bits coincida con el total de la dirección
4	Decodificación numérica de una dirección concreta (0xABCDE8F8) en sus 3 campos
5(b,c,d)	Simulación de recorrido de matrices para hit rate real (no estimado), detección de aliasing entre arreglos por alineamiento de direcciones base múltiplo del tamaño de caché, y rediseño de caché (aumentar asociatividad) para resolverlo
6(b,d,e,f)	Transferencias a nivel de bloque entre niveles de jerarquía de caché, y cómputo de ciclos de bus según ancho de bus

Para el examen: la jerarquía de memoria / caché es, de los temas teóricos “cortos”, el que más aparece en los parciales reales de este curso (9 de 13 exámenes analizados en la guía temática). El patrón de examen más importante de este práctico es el Ejercicio 5: **no basta con mirar si el recorrido coincide con el orden de almacenamiento** (localidad espacial “de libro”) — con correspondencia **directa** y varios arreglos del mismo tamaño, hay que revisar además si las **direcciones base** de esos arreglos están separadas por un múltiplo del tamaño de la caché, porque eso causa que arreglos distintos se “pisen” en la misma línea en cada acceso, arruinando el hit rate sin que el código en sí tenga ningún problema de localidad. La solución de fondo a ese tipo de conflicto no es cambiar el orden de recorrido ni el layout de almacenamiento, sino aumentar la asociatividad (agregar “vías”) para que las líneas en conflicto puedan convivir simultáneamente. Siempre conviene verificar los cálculos de bits sumando TAG+LÍNEA/CONJUNTO+BYTE contra el total de la dirección, y en ejercicios de jerarquía multinivel (Ejercicio 6), recordar que **cada nivel de caché tiene su propio tamaño de línea** deter-

minado por su propia capacidad, y que los pedidos entre niveles se hacen a nivel de bloque (no de palabra) — un miss en L1 mueve el bloque completo de L1, y si ese bloque tampoco está en L2, L2 rellena su **propio** bloque (más grande) desde memoria.